Library Functions in C

- C provides a collection of library functions for programmers
- Library functions are inbuilt functions in C programming
- If these library functions are used in a program, be sure to include their prototypes
- Library function prototypes are placed in specific header files

Library Functions contd..

- to use the printf() function, the header file <stdio.h> should be included.

Example: Square root using sqrt() function

Suppose, you want to find the square root of a number.

To compute the square root of a number, you can use the sqrt() library function. The function is defined in the math.h header file.

```
#include <stdio.h>
 #include <math.h>
int main()
 {
   float num, root;
printf("Enter a number: "); scanf("%f", &num);
// Computes the square root of num and stores in root
 root = sqrt(num);
printf("Square root of %.2f = %.2f", num, root); return 0;
```

Library Functions contd..

- When you run the program, the output will be:
- Enter a number: 12 Square root of 12.00 = 3.46

Library Functions in Different Header Files

C Header Files	functions	Example
<ctype.h>	Character type functions	*isupper(c), islower(c)*
<math.h>	Mathematics functions	*sin(d),cos(d), pow(d1,d2)*
<stdio.h>	Standard Input/Output functions	*scanf(…)*
<stdlib.h>	Standard Utility functions	*abs(i),rand()*
<string.h>	String handling functions	*strlen(s), strcpy(s1,s2) –copy string s2 to string s1*
<time.h>	Date time functions	*time(p), difftime(l1,l2)*

rand()

- The rand() function is used in C/C++ to generate pseudo-random numbers in the range [0, RAND_MAX).
- RAND_MAX: is a constant whose default value may vary between implementations but it is at least 32767.
- srand() sets the seed which is used by rand to generate “random” numbers. If you don’t call srand before your first call to rand, it’s as if you had called srand(1) to set the seed to one.
- srand() — Set Seed for rand() Function.

Floating point random number

```
float x = ((float)rand()/(float)(RAND_MAX)) * a;
```

generates floating point random number value between 0.0 and a.

Example to generate floating point random number

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[])
{
    srand((unsigned int)time(NULL));

    float a = 5.0;
    for (int i=0;i<20;i++)
        printf("%f\n", ((float)rand()/(float)(RAND_MAX)) * a);
    return 0;
}
```

Example to generate floating point random number contd..

- Output

- 1.625741
- 3.832026
- 4.853078
- 0.687247
- 0.568085
- 2.810053
- 3.561830
- 3.674827
- 2.814782
- 3.047727
- 3.154944
- 0.141873
- 4.464814
- 0.124696
- 0.766487
- 2.349450
- 2.201889
- 2.148071
- 2.624953
- 2.578719

Example to generate random numbers

```
// C program to generate random numbers
#include <stdio.h>
#include <stdlib.h>
#include<time.h>

// Driver program
int main(void)
{
    // This program will create different sequence of
    // random numbers on every program run

    // Use current time as seed for random generator
    srand(time(0));

    for(int i = 0; i<4; i++)
        printf(" %d ", rand());

    return 0;
}
```

Example to generate random numbers contd..

- Output 1:
- 453 1432 325 89
- Output 2:
- 8976 21234 45 8975
- ……..
- Output n:
- 563 9873 12321 24132
- Observe that each time different numbers are generated

time.h

- The time.h header file contains definitions of functions to get and manipulate date and time information.
- clock() This function returns the processor time consumed by a program
- ctime() This function returns the date and time in the format

  day month hours:minutes:seconds year

  Eg: Sat Oct 24 11:26:03 2021

Example to print time taken by a program

```
#include <stdio.h>
#include <time.h>
main ()
{
    time_t start, end;
    start = time(NULL);
    int a, b;
    scanf("%d %d", &a, &b);
    printf("Sum of %d and %d is %d\n",  a, b, a + b);
    end = time(NULL);
    printf("Time taken to print sum is %.2f seconds",
          difftime(end, start));
}
```

time_t is a data type in time.h

Example using clock() function

```
#include <math.h>
#include <stdio.h>
#include <time.h>
int frequency_of_primes(int n)
{
    // This function checks the number of
    // primes less than the given parameter
    int i, j;
    int freq = n - 1;
    for (i = 2; i <= n; ++i)
        for (j = sqrt(i); j > 1; --j)
            if (i % j == 0) {
                --freq;
                break;       }
    return freq;
}
```

Example using clock() function contd..

```
int main()
{
    clock_t t;
    int f;
    t = clock();
    f = frequency_of_primes(9999);
    printf("The number of primes lower"
          " than 10, 000 is: %d\n", f);
    t = clock() - t;
    printf("No. of clicks %ld clicks (%f seconds).\n",
          t, ((float)t) / CLOCKS_PER_SEC);
    return 0;
}
```

Output:

The number of primes lower than 10, 000 is: 1229

No. of clicks 2837 clicks (0.002837 seconds).

clock_t is a data type in time.h

CLOCKS_PER_SEC is a macro that gives the system clock ticks per second.

Output of Example to print time taken by a program

- Output:
- Sum of 4196144 and 0 is 4196144
- Time taken to print sum is 0.00 seconds

Sizeof () operator

- Unary operator returns the amount of memory allocated to the operand

Example to print the memory allocated in bytes to character, integer and float data types

```
# include <stdio.h>
main()
{
  int i;
  char c;
  printf (“size of i is % d bytes”, sizeof(i));
  printf (“\n size of c is % d bytes”, sizeof(c));
  printf (“\n size of float data type is  % d bytes”,sizeof(float));
```

Output: size of i is 2 bytes

size of c is 1 bytes

size of float data type is 4 bytes

Control Statements

- Control statement *if* is used for decision making
- Simple if statement
- if-else statement
- switch statement
- conditional operator statement
- goto statement

Simple if statement

- The general form is

```
if (condition)
  {
   statement-block
  }
  statement-x
```

Statement block may be a single statement or a group of statements

If condition is true, statement-block will be executed and after that statement-x will be executed; otherwise i.e., if condition is false, only statement-x will be executed

Example: Simple if statement

```
/* find ratio of integers*/
#include <stdio.h>
int main()
 {
   int a, b, c, d;
   float ratio;
 printf("Enter four integer values: ");
 scanf("%d %d %d %d", &a, &b, &c, &d);
 if (c-d !=0) /* Execute statement-block*/
  {
      ratio = (float) (a+b) /(float) (c-d);
      printf("Ratio = %f\n",ratio);
   } //end of if
return 0;
}  //end of main
```

output: Enter four integer values 12 23 34 45 Ratio = -3.181818

If - else statement

The general form is

```
if (condition)
  {
    statement-block1
  }
   else
   {
    statement-block2
   }
   statement-x
```

Statement block1 and statement-block2 may be a single statement or a group of statements

If condition is true, statement-block1 will be executed and control goes to statement-x otherwise i.e., if condition is false, statement-block2 will be executed and next statement i.e., statement-x will be executed

Example for if - else statement

```
/* Guess a number program*/
#include <stdio.h>
main()
 {
   int number = 100;
   int guess;
   printf (“Enter your guess for the number: \n”);
   scanf (“%d”, &guess);
   if (guess == number)
     printf (“\n Correct guess”);
  else
    printf (“\n Sorry! Wrong guess”);
}
```

Output: Enter your guess for the number: 200

Sorry! Wrong guess

Nested if – else statement

- When more than one if-else statement is involved, it is known as nested if-else statement
- Nested if-else is used when more than one decision is to be taken

The general form is

```
if (condition 1)
     statement 1;
else if (condition 2)
    statement 2;
 else if (condition 3)
    statement 3;
 statement x
```

Example of nested if – else statement

```
/* Finds the largest of three numbers*/
 #include <stdio.h>
 main()
 int a = 5; b = 2; c = 7;
If (a > b)
{
   if (a > c)
     printf (“ a is greatest”);
  else
     printf (“ c is greatest”);
  }
  else  /* it means  b > = a */
   {
        if (b> c)
          printf (“ b is greatest”);
       else
     printf (“ c is greatest”);
  }
}
```

Output: c is greatest

Switch statement

- Switch statement is an alternative to nested if-else statement
- The switch statement tests the value of a given variable or an expression against a list of case values, and

  when a match is found, a block of statements associated with that statement are executed

General form of switch statement

```
switch (expression)
{
   case constant_1:
      statements;

   case constant_2:
         statements;
            ……..
   case constant_n:
         statements;
  default:
         statements;
}
```

First the expression is evaluated to a value

The value is compared one by one with constant_1, constant_2, …, constant_n

The statements following the correct match are executed

If none of the cases match, the statements following default are executed

The default is optional, and if not present, no action takes place

Example for switch statement

```
/* Test whether a number is even or odd */
#include <stdio.h>
main()
{
  int n, ch
printf (“Enter the number: \n”);
scanf (“%d”, &n);
if (n%2 ==0)
ch  = 1;
else ch = 2;
switch (ch)
{
  case 1;
   printf (“number is even: \n”);
   break;
  case 2;
   printf (“number is odd \n”);
  }
 }
```

Output: Enter the number: 20

number is even

break is a keyword and break statement inside a loop terminates the loop and the program control resumes at the next statement following the loop

It can be used to terminate a case in the switch statement

Exercises

Write and execute C programs for the following

1. Accept an integer number called items between 1 and 5 that represents the number of items served in a lunch.

If the items are 5, display lunch is excellent.

If the items are 4, display lunch is very good.

If the items are 3, display lunch is good.

If the items are 1 or 2, display simple.

2. Accept an integer code and display designations as below

If the code is 1, display “professor”

If the code is 2, display “Associate professor”

If the code is 3, display “ Assistant professor”

Otherwise display not a valid code

3. Accept two integers and arithmetic operator to perform arithmetic operations

Note: Use if-else and/or switch case

Conditional operator ?

- The general form is

conditional expression ? Expression1: expression 2

The conditional expression is evaluated first. If the value is non zero, expression 1 is evaluated; otherwise, expression 2 is evaluated

result = (average_marks >=50) ? 1: 0

Goto Statement

The general form is

goto label

……

……

……

label: statement

label can be

a number or

a name

Example program

```
#include <stdio.h>
 #include <math.h>
  main()
  {
 double x, y;
  read:   scanf (“%f”, &x);
      if (x < 0) goto read;
     y = sqrt(x)
    printf (%f   %f\n”, x y)
     goto read;
    }
```

Iteration with loops

- When a number of statements are to be executed certain number of times, or until a condition is satisfied, iteration is performed using loops
- There are three types of loops in C
- for loop
- While loop
- Do while loop

For loop

- The general form of for loop is

```
for (initialization;  end condition; increment)
  {
    statement 1;
     statement 2;
     ……..
      statement n;
}
```

Execution of all statements once is called one iteration

- The initialization is with an assignment statement that is used to set the loop control variable
- The end condition is a relational expression that determines when the loop to exit
- The increment defines how the loop control variable will change each time the loop is repeated
- The above three are sepearated with a semicolon
- The for loop will be executed as long as the condition holds true
- Once the condition becomes false, the for loop will be ended and the program execution will resume on the statement following the for block

Example: for loop

```
/* program to print a message 5 times*/
 #include <stdio.h>
  main()
  {
    int i;
    for (i=1; i<=5; i++)
   {
   printf ("\n Iteration %d", i);
  } /* end for*/
 }   /* end main*/
```

Output: Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5

Features of for loop

- More than one variable can be initialized

for example the statements: p=1;

for (n=0; n<17; ++n) can be rewritten as

```
for (p=1,n=0; n<17; ++n)
```

- The increment section also can have more than one part as below

```
for (n=1, m=50; n<=m; n=n+1, m=m-1)
  {
    v = m/n;
    printf("%d %d %d\n"), n, m,v);
}
```

Features of for loop contd..

- The test condition may have any compound relation and the testing need not be limited only to the loop control variable as shown below

```
for (i=1, sum=0; i<20 && sum<100; ++i)
   {
      sum = sum + i;
      printf (“%d\n”, sum);
   }
```

As long as i<20 and sum <100 is true the for loop is executed

Either i>=20 or sum >= 100, the control will be out of the loop

Features of for loop contd..

- Expressions can also be allowed in the assignment statements of initialization and increment sections as shown below

```
for (x=(m+n)/2; x>0; x = x/2)
```

- One or more sections following for can be omitted as shown in the program segment below

```
m =5;
for(; m!=100;)
 {
  printf (“%d\n”, m);
 m = m+5;
 }
```

- To provide time delay, a null statement using for can be given as below
- For (j = 1000; j>0; j = j – 1);

The loop is executed 1000 times without any output; causes time delay

- for loop is used when we know the number of iterations beforehand

Effect of ++i and i++ in for loop

- If the return value of i is not used inside the loop as shown below, it makes no difference whether you use ++i or i++

```
 for (i=0;i<5; ++i)
  printf (“ \n Hello! %d”, i);
for (i=0;i<5; i++)
  printf (“ \n Hello! %d”, i);
```

- If the value as evaluated is used , then the type of increment is significant as shown below

```
   int n =0;
  for ( int i=0; n< 5; n = i++)
{
 …..
}
```

Example using for loop -1

```
Find the sum of digits of any integer number*/
 #include <stdio.h>
 main()
{
    int num, cnum, sumdigits, rem;
   sumdigits = 0;
  printf("\n Enter the number:");
  scanf("%d", &num);
  for (cnum = num; cnum>0; cnum/=10)
   {
     rem = cnum %10;
     sumdigits += rem;
   } /* end of for */
    printf ("\n Sum of digits = %d", sumdigits);
 } /* end of main */
```

output

```
Enter the number:2891
Sum of digits = 20
```

Example using for loop - 2

prints the following pattern

```
*
* *
* * *
* * * *
* * * *
```

```
/* Prints  as many stars as the row number */
/* the number of rows  are given as input */
 #include <stdio.h>
 main()
{
   int rows, i, j;
    printf("\n Enter the number of rows:");
  scanf("%d", &rows);
  for (i = 1;i<= rows; i++)
   {
     for (j = 1; j <=i; j++)
     printf("*");
    printf("\n");
   } /* end of for */

 } /* end of main */
```

Output

```
 Enter the number of rows : 5
The above pattern will be printed
```

The while loop

- while statement tests the condition in the beginning itself
- The general form of the while statement is

```
while (test condition)
{
  body of loop;
}
```

Body of loop includes one or more statements

As long as the test condition is true, the statements inside the while are executed; when the condition is false, control goes to the statement after the while loop

Example using while statement

```
/* C program to find sum of  squares of 10 natural numbers */
#include <stdio.h>
   main()
   {
     int n = 1, sum = 0;
     while (n <= 10)
     {
      sum = sum + n*n;
   } /* end of while */
  printf (“ sum =  %d\n”, sum);
 } /* end of main */
```

Example 2 using while

- To avoid executing the program again and again for testing with different data, while can be used as below

```
#include <stdio.h>
 main()
{
   char ch;
   while (ch !='y')
    ch = getchar();
}
```

Exercise 1: Compute average of any N numbers using while

Exercise 2: Evaluate y = x to the power of n (where n is a non negative integer) using while

The do-while statement

- In this the condition is tested at the end
- The general form of do-while is

```
do
 {
  body of do-while
 }
 while (test-condition)
```

- body of do-while may have one or more statements

Example 1 using do-while

```
/* Find the factorial of any number */
#include <stdio.h>
  main ()
   {
   int n, number, fact =1;
   printf (“Enter a positive number:”);
  scanf(“%d”, &n);
 number = n;
  do {
        fact *= n;
         n = n – 1; }
        while (n > 0);
     printf (“ Factorial of %  d =  %  d”, number, fact);
}
```

output

```
Enter a positive number : 6
Factorial of 6 = 720
```

Example 2 using do-while

```
/* Multiplication Tables*/
#include <stdio.h>
#define COLMAX 10
#define ROWMAX 12

 main()
{  int row, column, y;
    row = 1;
     do /* Outer loop begins */
    {
          column = 1;
           printf  ( " Table % d ", row);
   do  {/* Inner loop begins */
      {
     y= row*column;
    printf("%4d",y);
    column = column+1;
    }
 while (column <=COLMAX /* Inner loop ends*/
     printf ("\n");
     row = row + 1;
 }
  whiile ( row <=  ROWMAX);  /* Outer loop ends */
}
```

Output

```
Table  1       1    2    3    4    5    6    7    8     9     10

Table  2       2    4    6    8   10   12   14   16    18    20

……..

Table  12      12   24   36   48   60   72   84   96   108   120
```

Break and continue statements

- *break* statement is used to terminate loops or to exit from a switch
- When a *break* is encountered inside any loop, the control automatically passes to the statement after the loop
- It can be used within a *while*, a *do-while*, a *for* or a *switch* statement
- It is written as *break;*

Continue statement

- *continue* statement is used to bypass the remainder of the current pass through a loop
- When a *continue* statement is encountered, the remaining loop statements are skipped and the computation proceeds directly to the next iteration through the loop
- It can be used within a *while*, a *do-while* or a *for* statement
- It is written as *continue;*

Exit () function

- The exit() function is used to terminate the execution of a C program
- It is a standard library function in stdlib.h
- The general form is
- void exit(int status);
- status − value returned to the parent process.
- This function does not return any value.
- status is a value returned to the operating system after the termination of the program
- Value zero indicates that the termination is normal where as value one indicates error

Difference between exit(0) and exit(1)

BASIS FOR COMPARISON	EXIT(0)	EXIT(1)
Purpose	Reports the operating system about the "successful/normal" termination/completion of the program.	Reports the operating system about the "abnormal" termination of the program.
Syntax	exit(0);	exit(1);
Indicates	It indicates that the task has been successfully performed.	It indicates that the task has been aborted in between because of the error.
Macros	EXIT_SUCCESS	EXIT_FAILURE

Another example of exit(0) and exit(1)

- #include <stdlib.h> // standard library function
- int main ()
- {
- FILE * ptrFile;
- ptrFile = fopen ("myfile.txt","r");
- // open the file in read only mode
- if (ptrFile==NULL)
- {
- cout<<"Error in opening file";
- exit (1); //alternately you can use exit(EXIT_FAILURE)
- }
- exit (0); //alternately you can use exit(EXIT_SUCCESS)
- }

Example program with exit(0)

```
#include <stdio.h>
#include <stdlib.h>

int main () {
   printf("Start of the program....\n");

   printf("Exiting the program....\n");
   exit(0);

   printf("End of the program....\n");

   return(0);
}
```

Output

Start of the program....

Exiting the program....